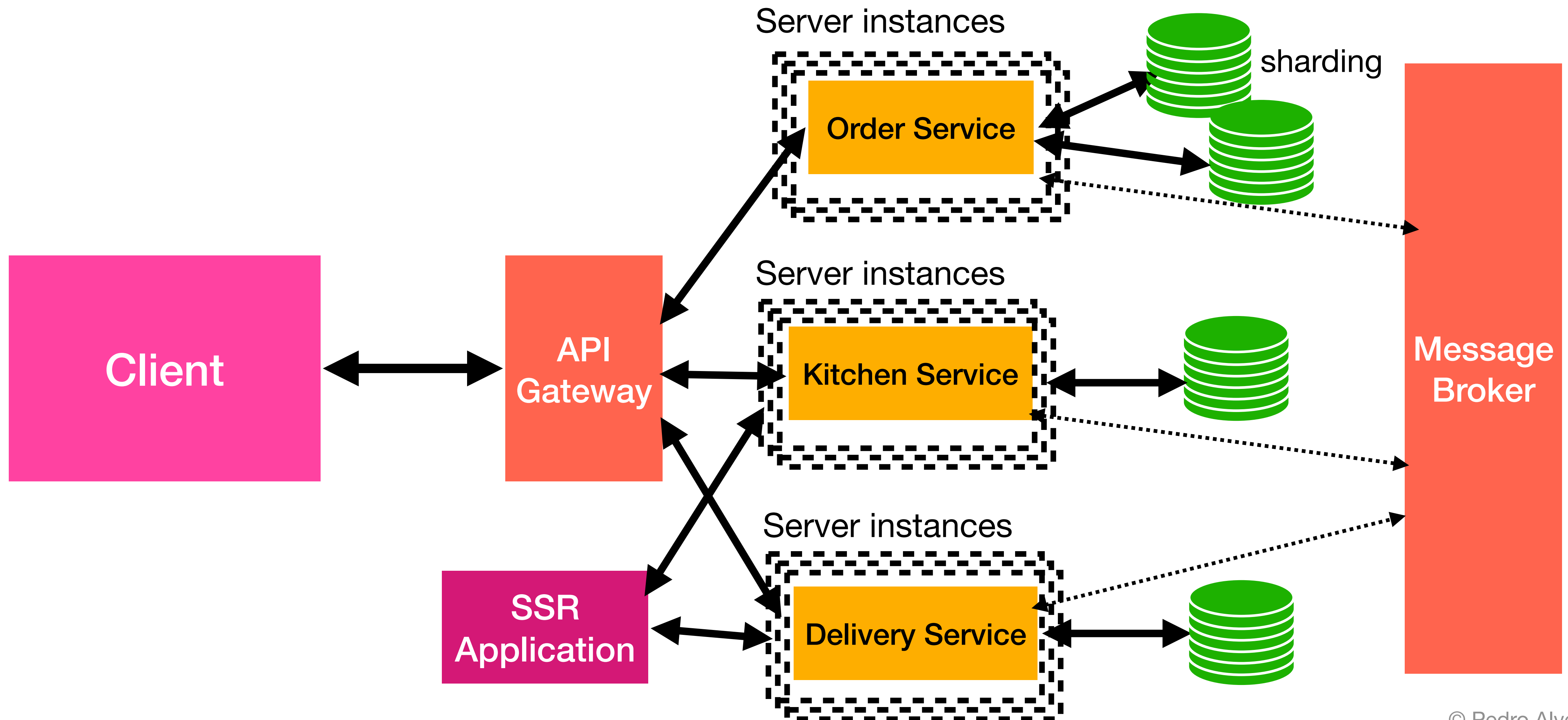


Internet Applications Design and Implementation

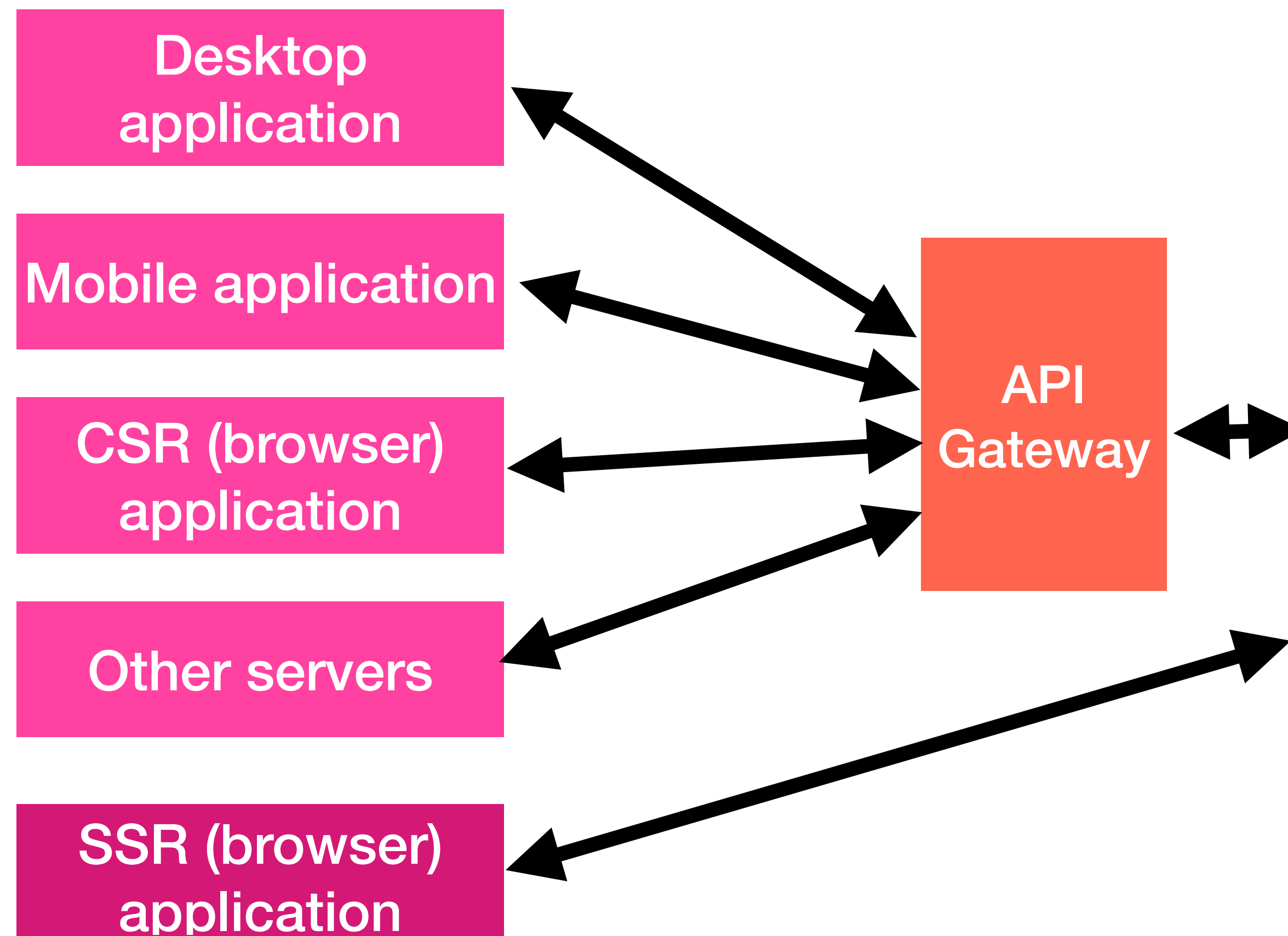
**Week 9 - Client-side rendering. Component-based web apps.
React.**

Service architecture

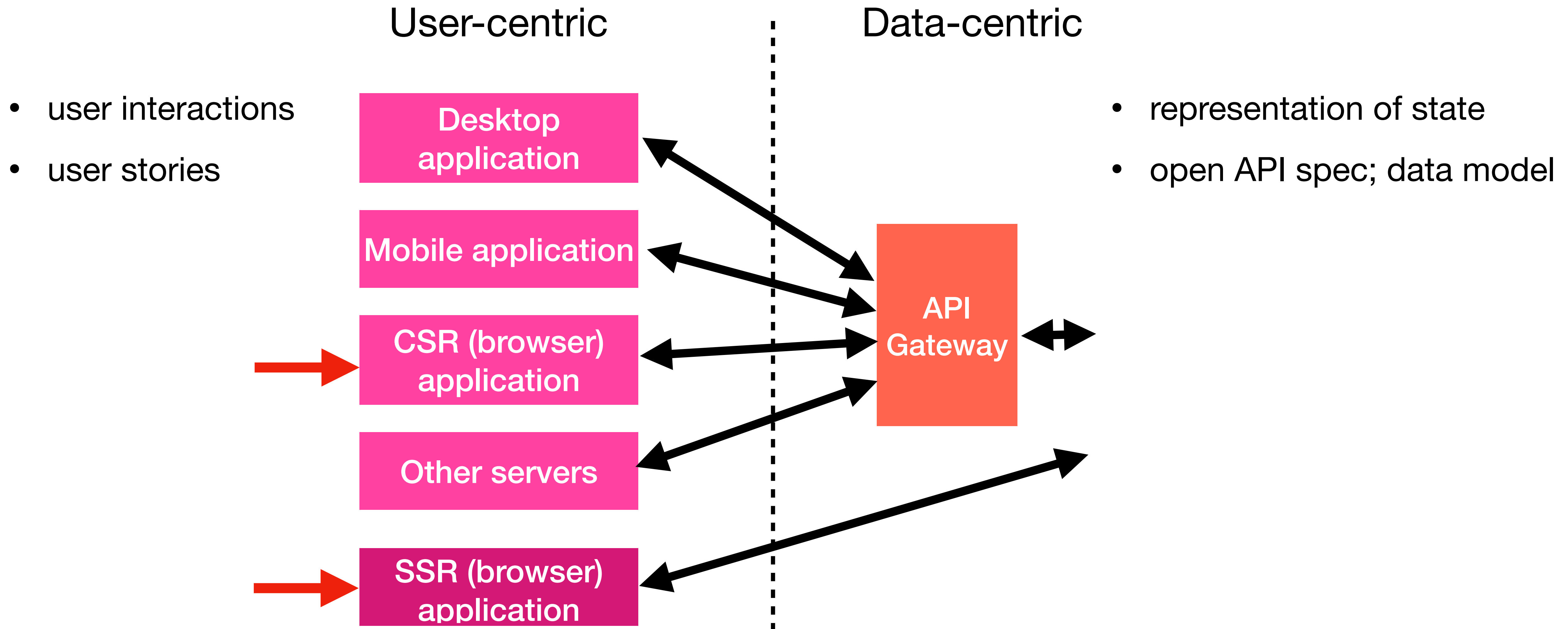
Revisions from lecture 5 & 7



Client types



User-centric vs Data-centric development



Rendering CSR vs SSR

Revisions

Here, **rendering** refers to the process of **generating HTML**, not the browser's visual display of that HTML on the screen.

Server-Side Rendering - The server generates the complete HTML for the page. The browser receives fully built HTML and displays it.

Client-Side Rendering - The browser receives minimal HTML (often just a root container) along with JavaScript. The JavaScript runs in the browser and dynamically builds the HTML.

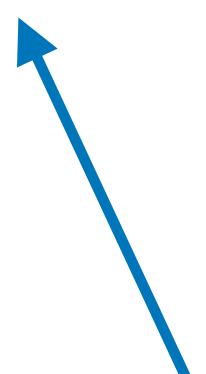
Rendering CSR vs SSR

Revisions

Client-side rendering

```
<html>
<body>
  <div id="root"></div>

  <script src="app.js"></script>
</body>
</html>
```



Fetch data from the server
and build the HTML in the
browser

Server-side rendering

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <body>
    <div id="root">
      <h1 th:text="${event.name}">Event Name</h1>

      <p>Date: <span th:text="${event.date}" />
      <p>Location: <span th:text="${event.location}" />
    </div>
  </body>
</html>
```

(example using thymeleaf)

Rendering CSR vs HTMX

Revisions

Client-side rendering

```
<!-- index.html -->
<div id="users"></div>

<script>
  fetch('/users')
    .then(res => res.json())
    .then(data => {
      const container = document.getElementById('users');
      container.innerHTML = data.map(u => `<p>${u.name}</p>`)
        .join('');
    });
</script>
```

← returns JSON

HTMX + Thymeleaf

```
<!-- index.html -->
<div hx-get="/users" hx-trigger="load" hx-target="#users">
</div>

<div id="users"></div>

<!-- users.html (Thymeleaf fragment) -->
<div th:each="u : ${users}">
  <p th:text="${u.name}"></p>
</div>
```

← returns HTML (fragment)

Rendering CSR vs HTMX

Revisions

Client-side rendering

```
<!-- index.html -->
<div id="users"></div>

<script>
  fetch('/users')
    .then(res => res.json())
    .then(data => {
      const container = document.getElementById('users');
      container.innerHTML = data.map(u => `<p>${u.name}</p>`)
        .join('');
    });
</script>
```

← returns JSON

HTMX + Thymeleaf

```
<!-- index.html -->
<div hx-get="/users" hx-trigger="load" hx-target="#users">
</div>

<div id="users"></div>

<!-- users.html (Thymeleaf fragment) -->
<div th:each="u : ${users}">
  <p th:text="${u.name}"></p>
</div>
```

← returns HTML (fragment)

Rendering

SSR vs HTMX vs CSR

SSR

- Site has basic interactivity (e.g. CRUD)
- First page load needs to be fast
- Better for SEO
- Templates $\leftrightarrow$ Data needs to be checked in compile time

HTMX

- Site needs some interactivity without resorting to Javascript
- Needs to avoid Javascript tooling complexity (transpilers, bundlers, ...)

CSR

- Site needs rich interactivity (e.g., gmail, google docs, netflix, trello, ...)
- Site pages include multiple independent components (e.g., portals, social networks, ...)
- Company has a dedicated front-end team

Browser logic architecture

- Static Source
 - HTML (initial content)
- Browser representation (DOM)
 - Elements (structure)
 - Style (CSS)
 - Event handlers
- Runtime Behaviour
 - Javascript logic (dynamic updates)
- User Interface
 - Rendered view (what the users see)

HTML Evolution

- 1991 - HTML (Tim Berners-Lee)
 - Basic structure: heading, paragraphs, links
 - No styling
- 1995 - HTML 2.0 (first official standard)
 - Forms introduced
- 1997 - HTML 4.0
 - Separation of content & presentation (CSS introduced)
 - Scripting support (JavaScript integration)
- 2014 - HTML 5
 - Semantic tags (`<header>`, `<footer>`, `<article>`)
 - Native media (`<video>`, `<audio>`)
 - APIs (Canvas, Geolocation, Local Storage)

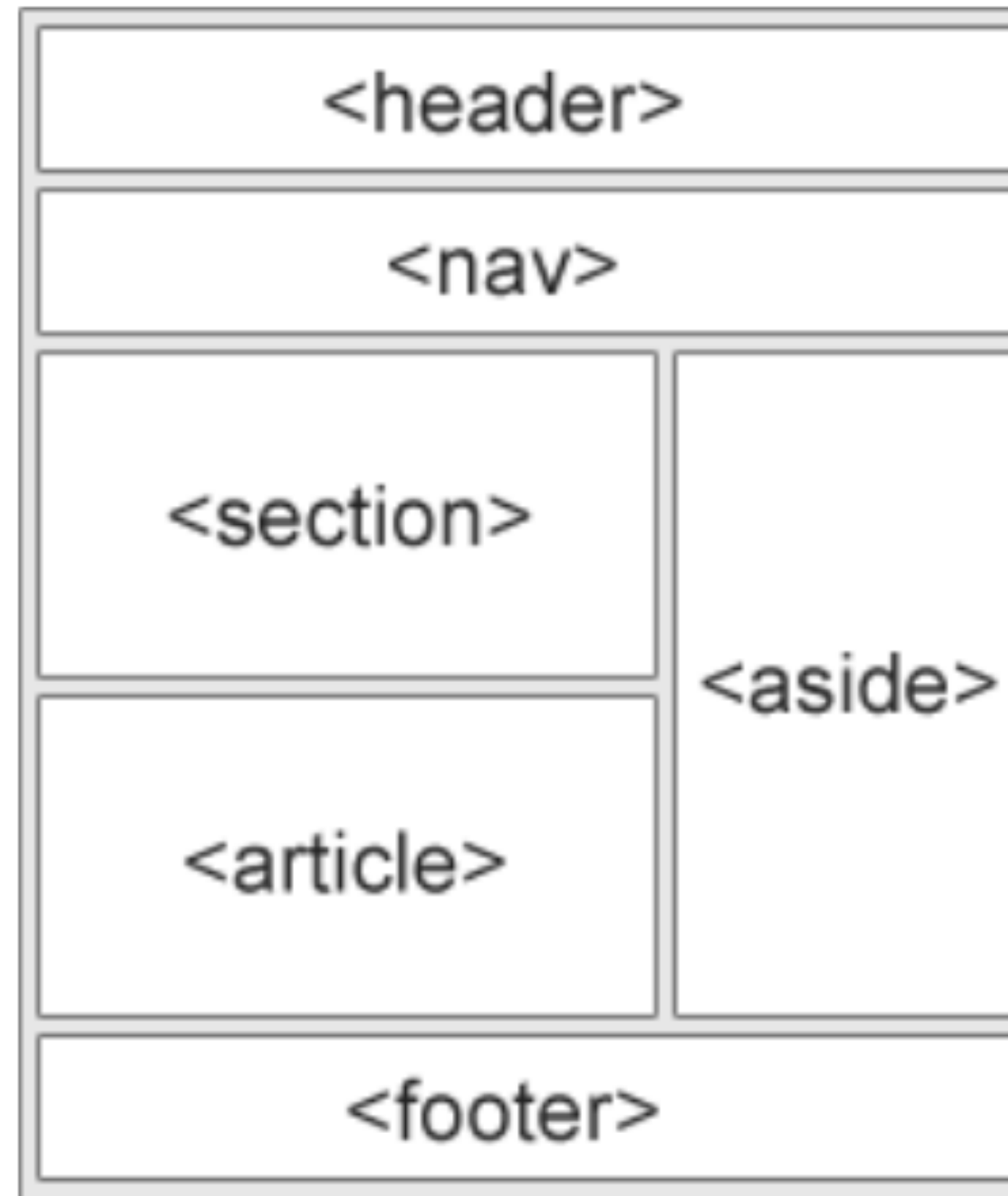
HTML

- Each HTML element has a tag
 - `<body>` `<ul>` `<li>` `<p>` ...
- Content
 - `<p>This is a paragraph</p>`
- and a set of attributes:
 - Generic: set on any HTML element. (ex: `id`, `class`, `hidden`, ...)
 - Particular: `img/src`, `option/value`, etc.
 - Events: dependent on the kind of element
 - `body / onload`, `button / onclick`, etc.

HTML essentials in one slide

- `<!DOCTYPE html>`
- `<html></html>` **root element**
- `<head></head>` **section to declare meta-information, stylesheets and scripts**
- `<body></body>` **section to define content**
- `<p>` `<h1>...` `<ul>` `<div>` **block elements**
 - Take up the full width available; start on a new line; stack vertically
- `<span>` `<strong>` `<i>` **inline elements**
 - Take only the space they need; stay in the same line; flow inside text
- `<`; `>`; `é` **escaped characters**

HTML 5 semantic elements



HTML 5 semantic elements

Tag	Description
<code><article></code>	Defines an article in the document
<code><figcaption></code>	Defines a caption for a <code><figure></code> element
<code><figure></code>	Defines self-contained content, like illustrations, diagrams, photos, code listings, etc.
<code><footer></code>	Defines a footer for the document or a section
<code><header></code>	Defines a header for the document or a section
<code><main></code>	Defines the main content of a document
<code><menuitem></code>	Defines a command/menu item that the user can invoke from a popup menu
<code><nav></code>	Defines navigation links in the document
<code><section></code>	Defines a section in the document

Typical HTML structure

```
<!doctype html> <!-- Declares HTML5 document -->
<html lang="en"> <!-- Root element, language = English -->

  <head> <!-- head defines invisible information -->
    <title>My Page</title> <!-- Title shown in browser tab -->

    <meta charset="utf-8">
    <meta name="viewport" content="initial-scale=1, width=device-width">
    <meta name="description" content="Some description">
    <meta name="robots" content="index, follow">

    <!-- External CSS stylesheet (controls layout & design) -->
    <link rel="stylesheet" type="text/css" href="/assets/css/application.css?v=1.0.6">

    <!-- External JavaScript file -->
    <script src="/assets/js/application.js?v=1.0.6" type="module"></script>
  </head>

  <body> <!-- Body defines visible content -->

    <!-- Navigation section (menus, links) -->
    <nav>
    </nav>

    <!-- Main content of the page -->
    <main>
    </main>

    <!-- Footer section (copyright, links, etc.) -->
    <footer>
    </footer>

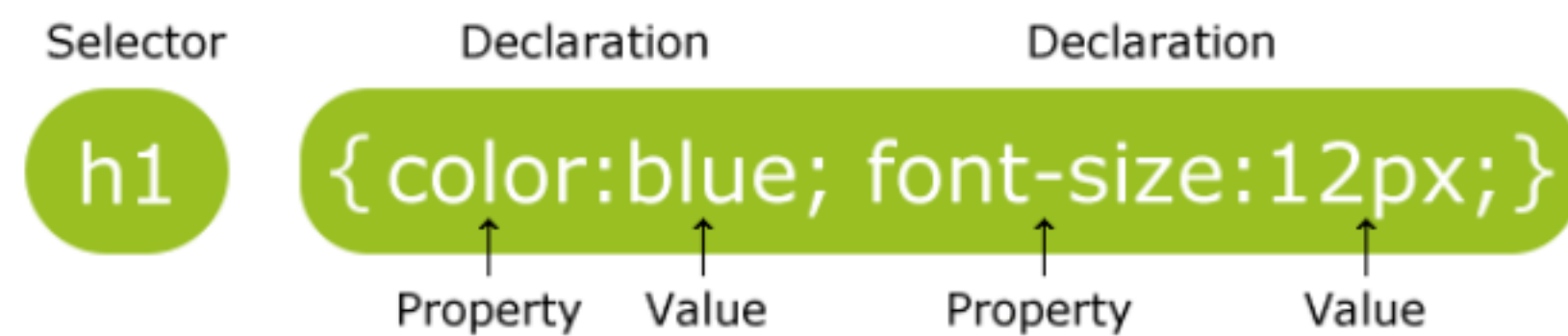
  </body>
</html>
```


CSS - Cascading Style Sheets

- Describes the presentation of a document defined using a markup language (HTML, XML)
- Decouples presentation from the document structure and behavior of a web application
- Based on declarative object descriptors and object attributes (and values) - Rules
- Based on general application priorities on conflicting cases
- Includes basic presentation behavior (animations)

CSS - Cascading Style Sheets

- A set of rules is applied to the DOM



- rule = selector + style definitions with the form property:value
- Rules are applied by the following order
 - Browser default
 - External and Internal Style Sheet (following the order in the section <head>)
 - Inline Style (HTML element)

CSS selectors

Selector	Example	Example description	CSS
<u>.class</u>	.intro	Selects all elements with class="intro"	1
<u>#id</u>	#firstname	Selects the element with id="firstname"	1
<u>*</u>	*	Selects all elements	2
<u>element</u>	p	Selects all <p> elements	1
<u>element,element</u>	div, p	Selects all <div> elements and all <p> elements	1
<u>element element</u>	div p	Selects all <p> elements inside <div> elements	1
<u>element>element</u>	div > p	Selects all <p> elements where the parent is a <div> element	2
<u>element+element</u>	div + p	Selects all <p> elements that are placed immediately after <div> elements	2
<u>element1~element2</u>	p ~ ul	Selects every element that are preceded by a <p> element	3
<u>[attribute]</u>	[target]	Selects all elements with a target attribute	2
<u>[attribute=value]</u>	[target=_blank]	Selects all elements with target="_blank"	2
<u>[attribute~=value]</u>	[title~=flower]	Selects all elements with a title attribute containing the word "flower"	2
<u>[attribute =value]</u>	[lang =en]	Selects all elements with a lang attribute value starting with "en"	2
<u>[attribute^=value]</u>	a[href^="https"]	Selects every <a> element whose href attribute value begins with "https"	3

Responsive design



Responsive design

- Sets of methods and techniques to optimize reading and navigation of a certain web design to a myriad of screen sizes / device conditions
- Usually achieved by gracious resizing, hiding, and rearrangement of page sections.
 - Media queries (CSS @media rule)
 - Flexible (flow and grid) layouts (e.g. bootstrap)
 - JavaScript reactive interfaces

Responsive design

The @media rule is used to define different style rules for different screen sizes

```
@media screen and (max-width: 479px) {  
    .main-wrapper { width: 300px; }  
}  
  
@media screen and (min-width: 480px) and (max-width: 767px) {  
    .main-wrapper { width: 460px; }  
}  
  
@media screen and (min-width: 768px) and (max-width: 1023px) {  
    .main-wrapper { width: 760px; }  
}  
  
@media screen and (min-width: 1024px) and (max-width: 1177px) {  
    .main-wrapper { width: 960px; }  
}  
  
@media screen and (min-width: 1178px) and (max-width: 1279px) {  
    .main-wrapper { width: 1150px; }  
}
```


Responsive design

Can also be used to differentiate based on device conditions

```
@media (min-width: 700px) { ... }
```

```
@media (min-width: 700px) and (orientation: landscape) { ... }
```

```
@media tv and (min-width: 700px) and (orientation: landscape) { ... }
```

```
@media (min-width: 700px), handheld and (orientation: landscape) { ... }
```

Responsive design

There are several popular open-source “libraries” for responsive design:

- Tailwind CSS - <https://tailwindcss.com/>
- Bootstrap - <https://getbootstrap.com/>
- Material UI - <https://www.muicss.com/>
- ...

Javascript in the browser

Inline

```
<button onclick="alert( 'Hi' )" >Click</button>
```

simple, leads to messy code

<script> inside HTML

```
<script>
  console.log( "Hello" );
</script>
```

embedded in the page, good for quick validations

External script (classic)

```
<script src="app.js"></script>
```

separation of concerns, runs in global scope,
may run before page is fully loaded

External script (module)

```
<script type="module" src="app.js"></script>
```

separation of concerns, runs in module
scope, deferred execution until page is fully
loaded

Javascript in the browser

Using event handlers in HTML elements

```
<h1>JavaScript Can Validate Input</h1>

<p>Please input a number between 1 and 10:</p>

<input id="number" type="number">

<button type="button" onclick="myFunction()">Submit</button>

<p id="demo"></p>

<script>
    function myFunction() {
        var x, text;

        // Get the value of the input field with id="numb"
        x = document.getElementById("number").value;

        // If x is Not a Number or less than one or greater than 10
        if (isNaN(x) || x < 1 || x > 10) {
            text = "Input not valid";
        } else {
            text = "Input OK";
        }

        document.getElementById("demo").innerHTML = text;
    }
</script>
```

Javascript in the browser

```
<h1>JavaScript Can Validate Input</h1>

<p>Please input a number between 1 and 10:</p>

<input id="number" type="number">

<button type="button" id="submitBtn">Submit</button>

<p id="demo"></p>

<script>
    function myFunction() {
        var x, text;

        // Get the value of the input field with id="numb"
        x = document.getElementById("number").value;

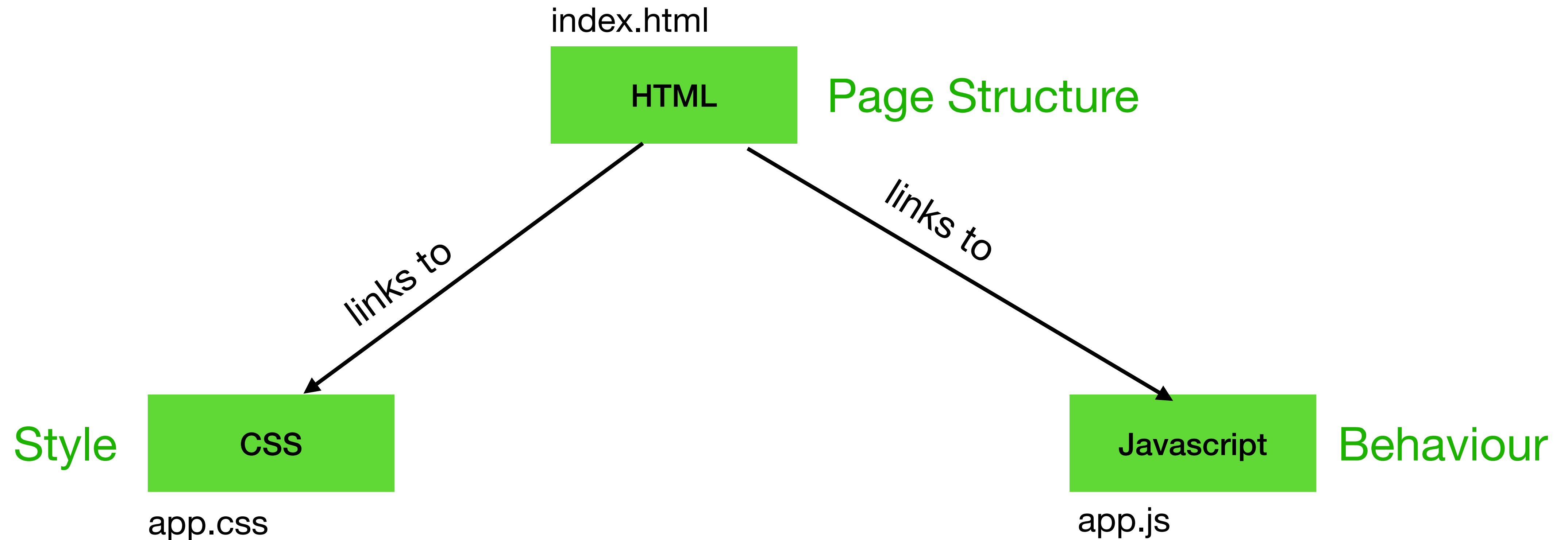
        // If x is Not a Number or less than one or greater than 10
        if (isNaN(x) || x < 1 || x > 10) {
            text = "Input not valid";
        } else {
            text = "Input OK";
        }

        document.getElementById("demo").innerHTML = text;
    }

    // Attach event listener instead of inline onclick
    document.getElementById("submitBtn").addEventListener("click", myFunction);
</script>
```

Using event listeners, HTML elements stay “clean”

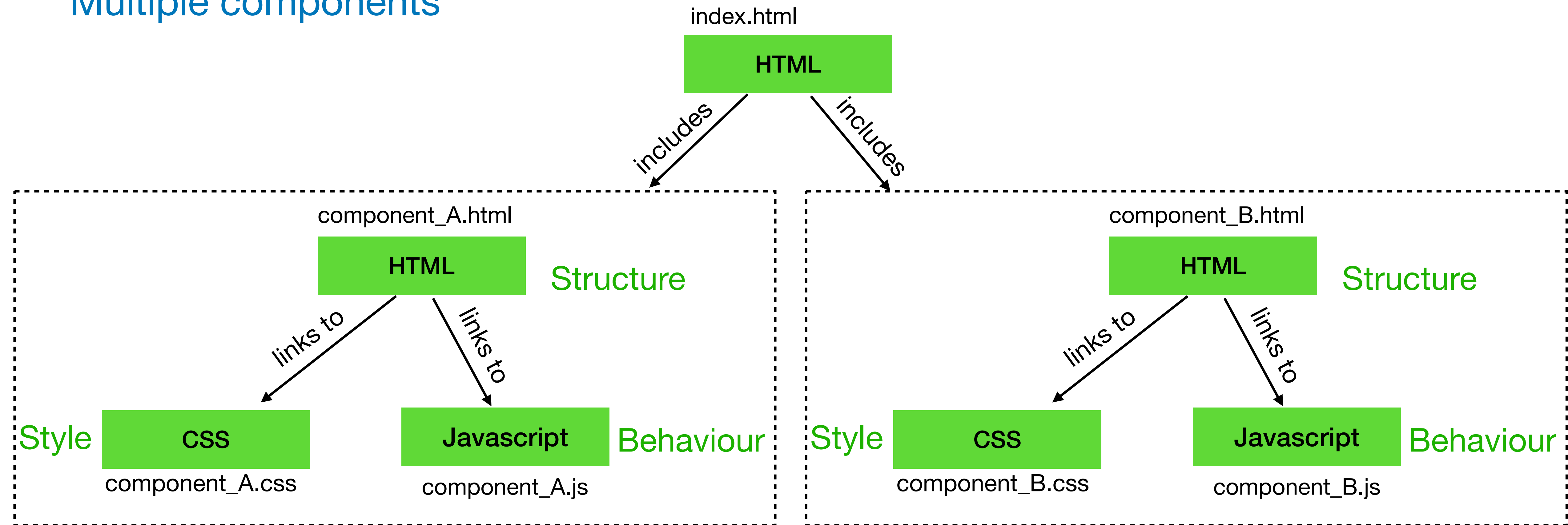
Standard web page



👍 Clear separation of concerns

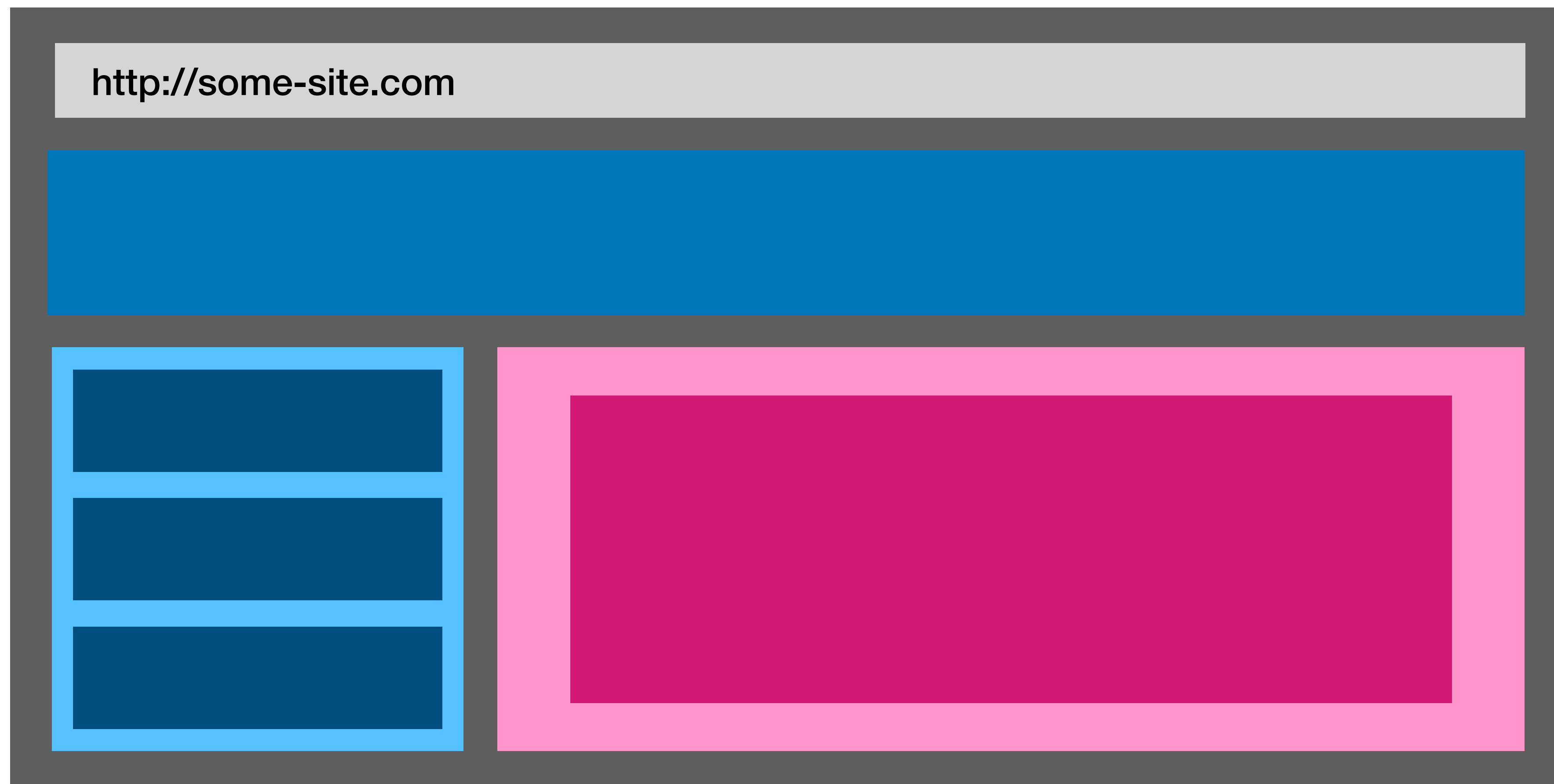
Component-based web page

Multiple components

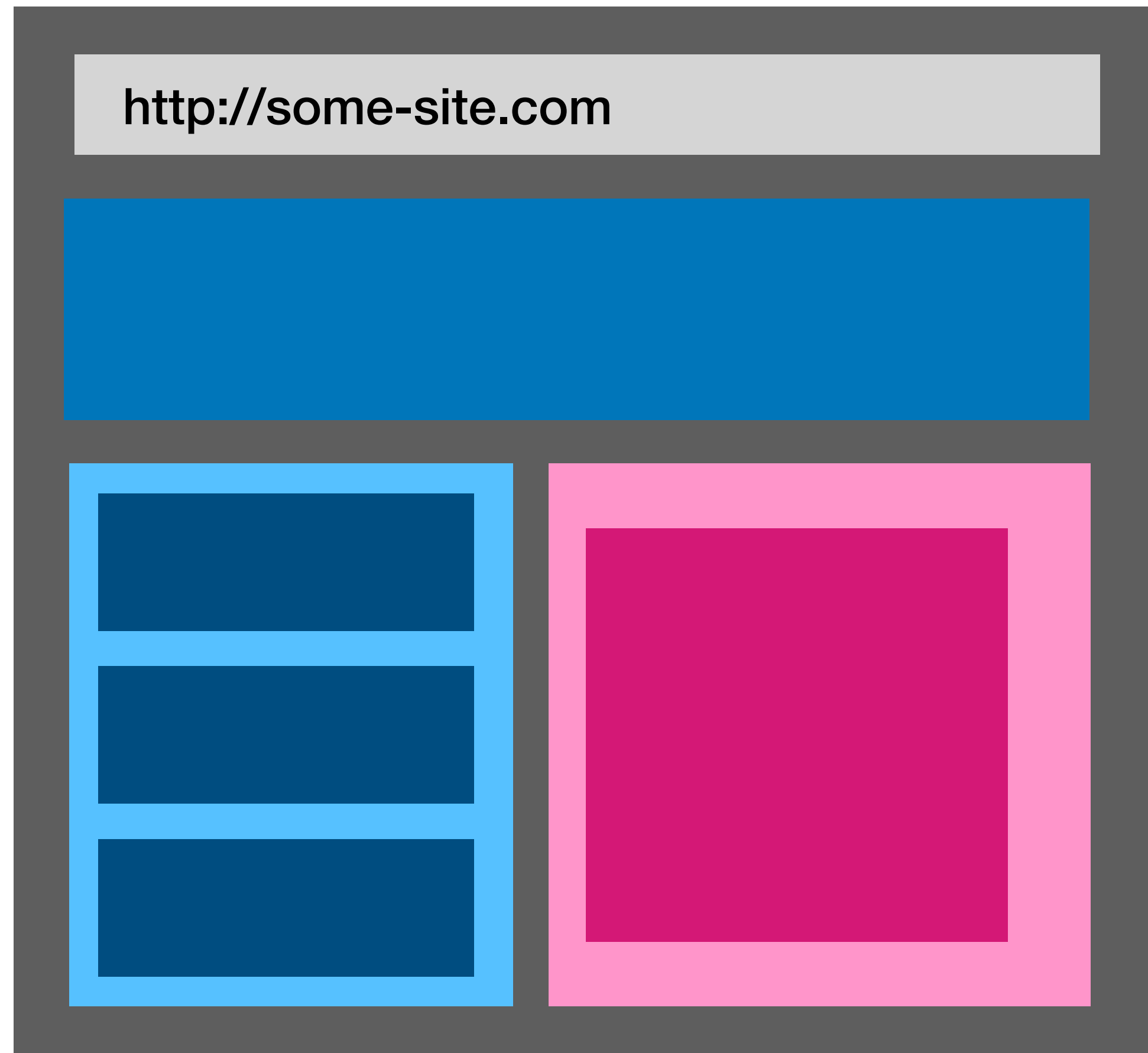


Component-based web page

Components are UI Building Blocks



Component-driven development



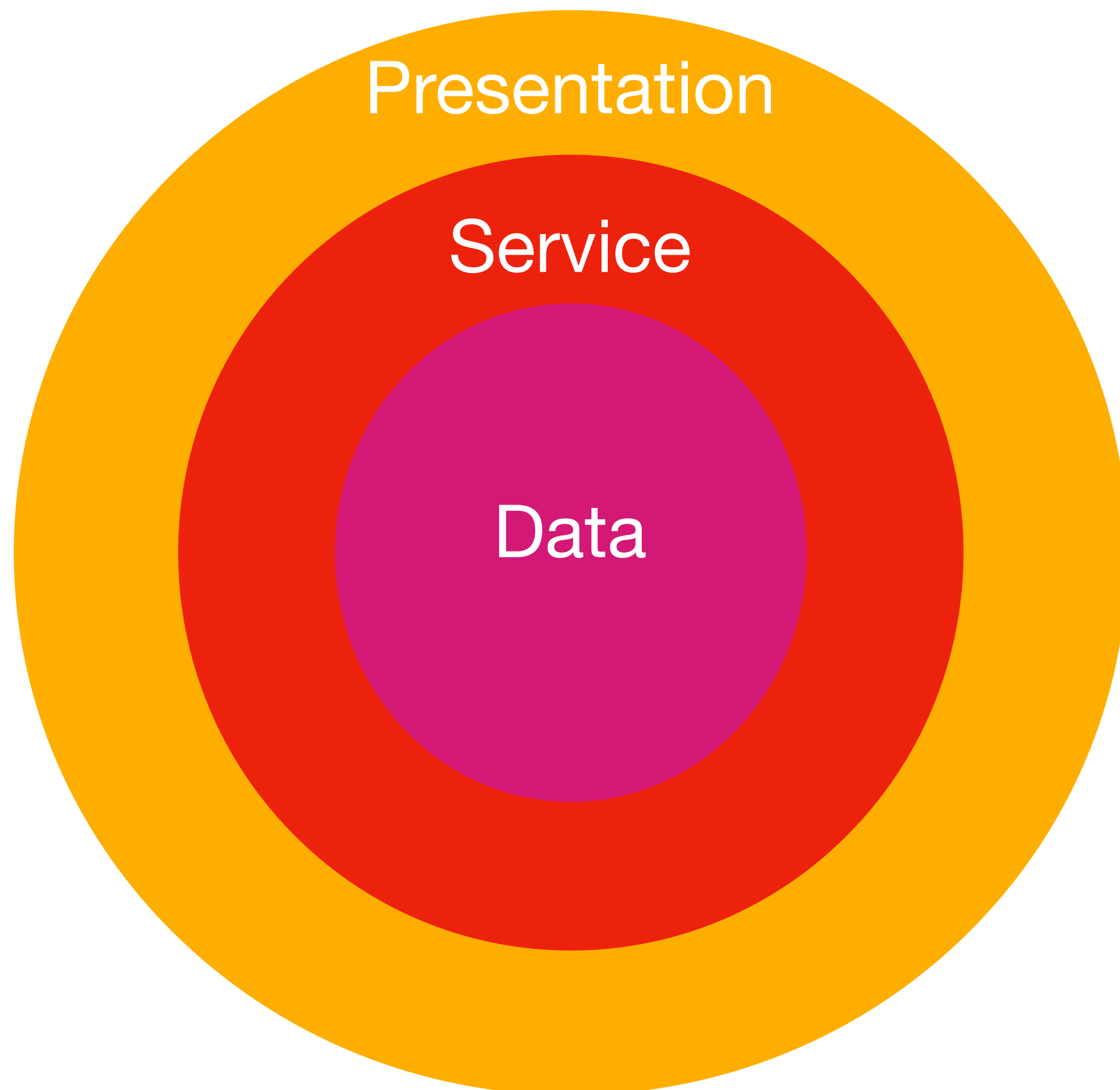
Components centralize the HTML, CSS and Javascript into single UI modules

Advantages

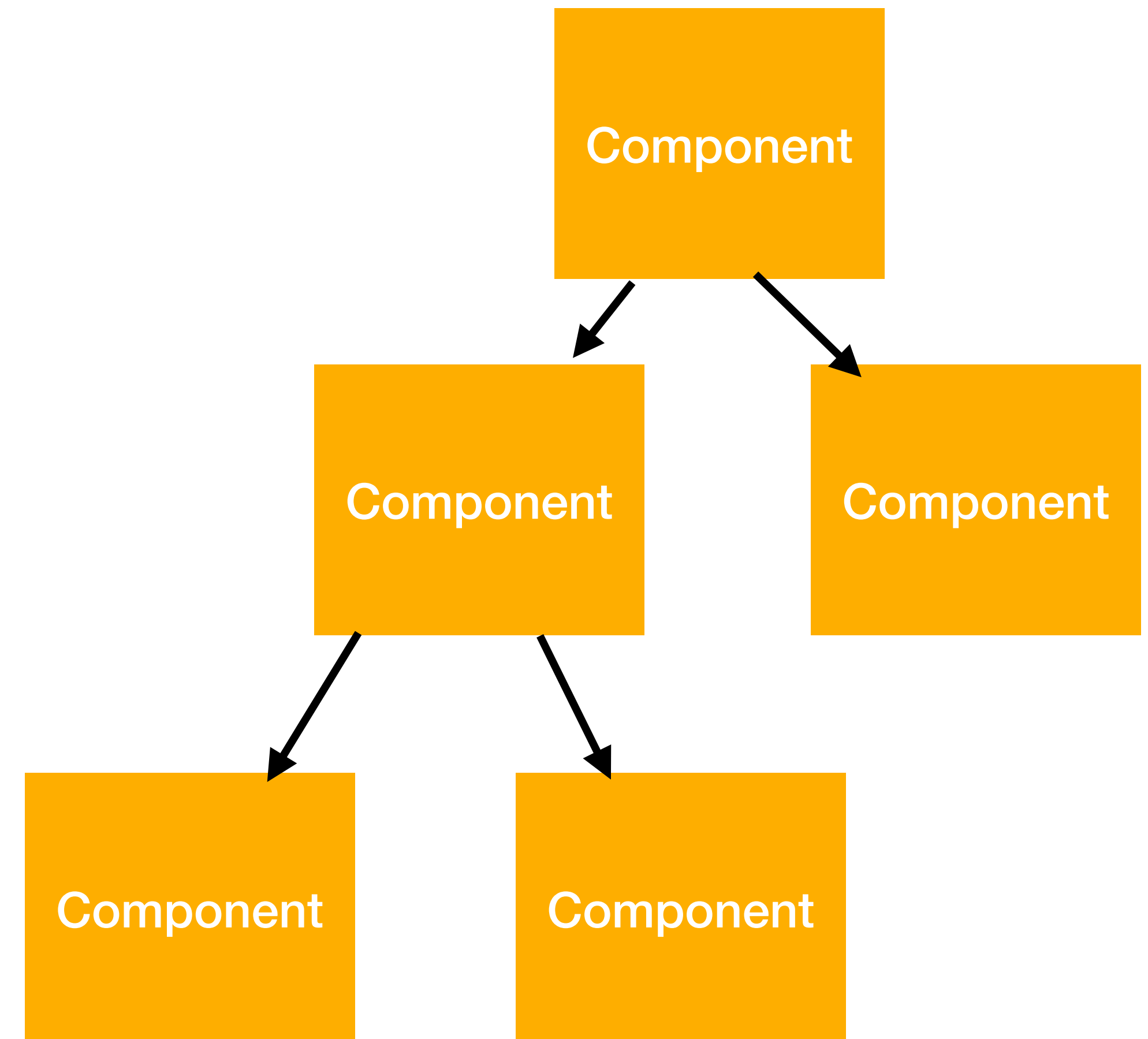
- Reusability
- Composability
- Performance

Decomposition

Layer decomposition



Component decomposition



Decomposition

Layer decomposition

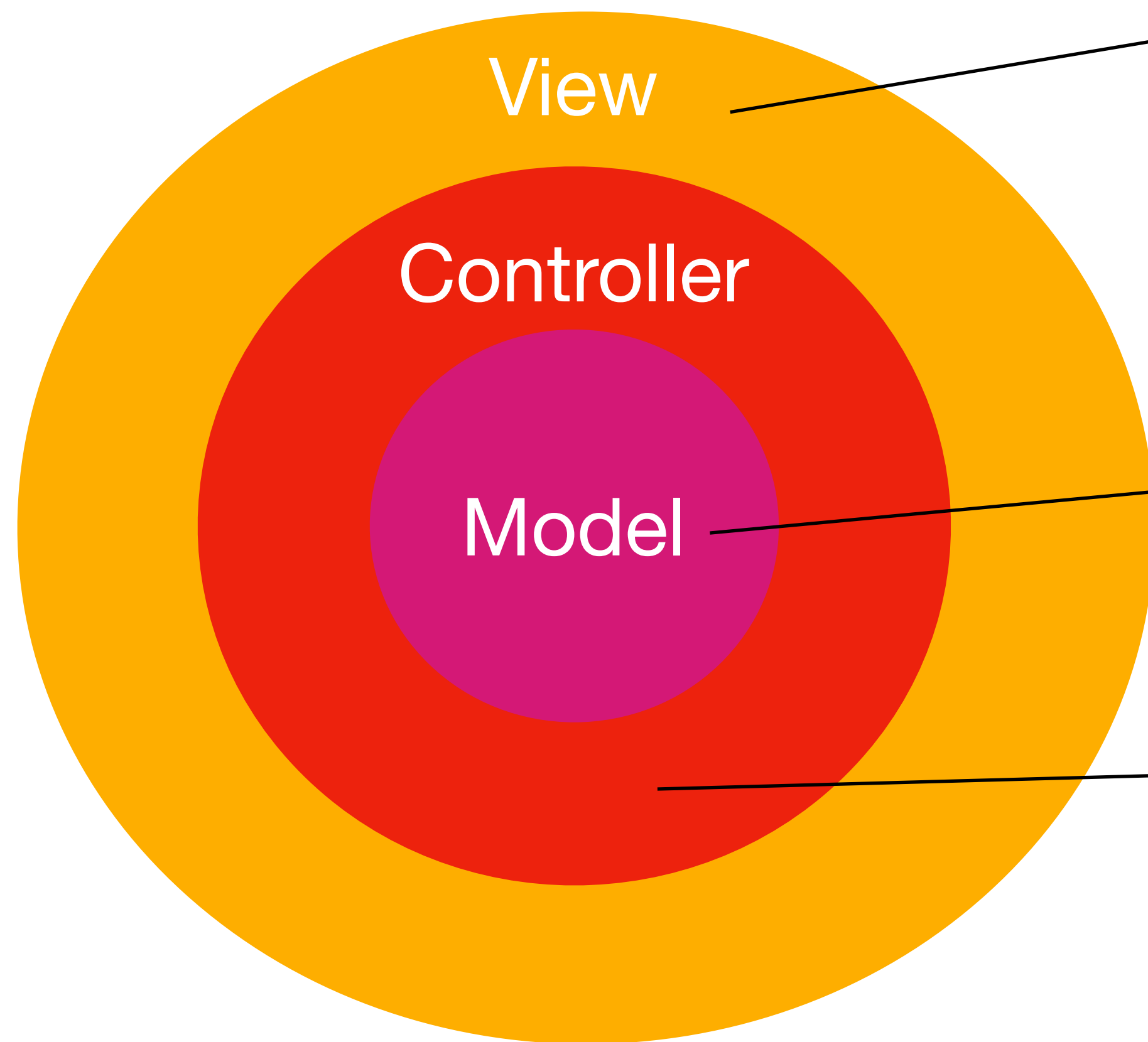
- Separation by technical concern
- Each layer has a single responsibility
- Pros: clear architecture, oriented towards request processing, good for backend
- Cons: features spread across layers, changing a single user feature touches several layers

Component decomposition

- Separation by feature
- Components are self-contained
- Pros: easy to reason about single features, better reusability
- Cons: technical concerns are mixed, hard to keep consistency across features

Technical Decomposition

MVC as Layer decomposition



```
<h1>JavaScript Can Validate Input</h1>

<p>Please input a number between 1 and 10:</p>

<input id="number" type="number"/>
<button type="button" id="submitBtn">Submit</button>
<p id="result"></p>
```

```
<script>
function validate(number) {
  return !isNaN(number) && number >= 1 && number <= 10;
}
```

```
document.getElementById("submitBtn")
  .addEventListener("click", () => {
    const number = Number(document.getElementById("number").value);

    document.getElementById("result").innerHTML =
      validate(number) ? "Input OK" : "Input not valid";
  });
</script>
```

Component decomposition

Reusability

amazon

Deliver to Pedro

All ▾ Search Amazon 🔍

EN Hello, Pedro Account & Lists Returns & Orders Cart 2

All Rufus Today's Deals Buy Again Browsing History Customer Service Pedro's Amazon.com Sell Registry Prime Video

Shopping Cart

Deselect all items

✓



Patterns of Enterprise Application Architecture

by Martin Fowler

Hardcover

In Stock

FREE International Returns

☐ This is a gift Learn more

Format: Hardcover

🗑️

1

+

Delete

Save for later

Compare with similar items

Share

✓



The Missing README: A Guide for the New Software Engineer

by Chris Riccomini

Paperback

In Stock

FREE International Returns

☐ This is a gift Learn more

Format: Paperback

🗑️

1

+

Delete

Save for later

Compare with similar items

Share

Subtotal (2 items): \$81.09

☐ This order contains a gift

Proceed to checkout

prime

Enjoy fast, FREE delivery on eligible items with a 30-day trial for \$0

Try Prime FREE

Books you may like



Project Hail Mary: A...

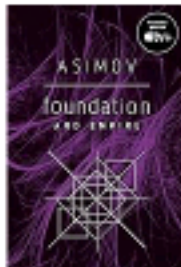
> Andy Weir

★★★★☆ 231,784

Kindle Edition

1 offer from \$13⁹⁹

See all buying options



Foundation and Empire

> Isaac Asimov

★★★★☆ 14,373

Kindle Edition

-5% \$8⁰⁵

Digital List Price: \$8.~~48~~

See all buying options

Subtotal (2 items): \$81.09

CardItem (reused 2x)

Component decomposition

Reusability

amazon

Deliver to Pedro

All ▾

Search Amazon

Q

EN ▾

Hello, Pedro

Account & Lists ▾

Returns & Orders

2 Cart

All

Rufus

Today's Deals

Buy Again

Browsing History ▾

Customer Service

Pedro's Amazon.com

Sell

Registry

Prime Video

Shopping Cart

[Deselect all items](#)

✓



Patterns of Enterprise Application Architecture

by Martin Fowler

Hardcover

In Stock

FREE International Returns

☐ This is a gift [Learn more](#)

Format: Hardcover

1

+

Delete

Save for later

Compare with similar items

Share

✓



The Missing README: A Guide for the New Software Engineer

by Chris Riccomini

Paperback

In Stock

FREE International Returns

☐ This is a gift [Learn more](#)

Format: Paperback

1

+

Delete

Save for later

Compare with similar items

Share

Subtotal (2 items): \$81.09

Subtotal (2 items): \$81.09

☐ This order contains a gift

Proceed to checkout

prime

Enjoy fast, **FREE** delivery on eligible items with a 30-day trial for \$0

Try Prime FREE

Books you may like



Project Hail Mary: A...

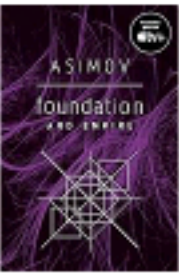
> Andy Weir

★★★★☆ 231,784

Kindle Edition

1 offer from \$13.99

See all buying options



Foundation and Empire

> Isaac Asimov

★★★★☆ 14,373

Kindle Edition

-5% \$8.05

Digital List Price: \$8.48

See all buying options

BookSuggestion
(reused 2x)

© Pedro Alves 2026

Component properties

Each component has a set of immutable and mutable properties that determine how it is rendered

Shopping Cart

[Deselect all items](#)

		Price
☒		Patterns of Enterprise Application Architecture by Martin Fowler Hardcover In Stock FREE International Returns ☐ This is a gift Learn more Format: Hardcover 🗑️ 1 + Delete Save for later Compare with similar items Share
☒		The Missing README: A Guide for the New Software Engineer by Chris Riccomini Paperback In Stock FREE International Returns ☐ This is a gift Learn more Format: Paperback 🗑️ 1 + Delete Save for later Compare with similar items Share
Subtotal (2 items):		\$81.79

Example: CardItem

Immutable properties

- Title
 - Author
 - Price
 - CoverImage
- Provided from outside the component
 - Do not change inside the component

Mutable properties

- Quantity
 - Checked
- Managed inside the component
 - Change over time (user interaction, data fetch, ...)

Stateless component

Stateless component is a component that only has immutable properties

Books you may like



BookSuggestion only has immutable properties:

- Title, Author, Rating, Price, CoverImage



This means it must be rendered only once

Stateful component

Stateful component is a component that has mutable properties



Mutable properties are also known as state

State starts with an initial value and changes over time (e.g., user interaction)

The component also handles interaction

Every change to the state must trigger a new rendering, since the view has become obsolete

Stateful component



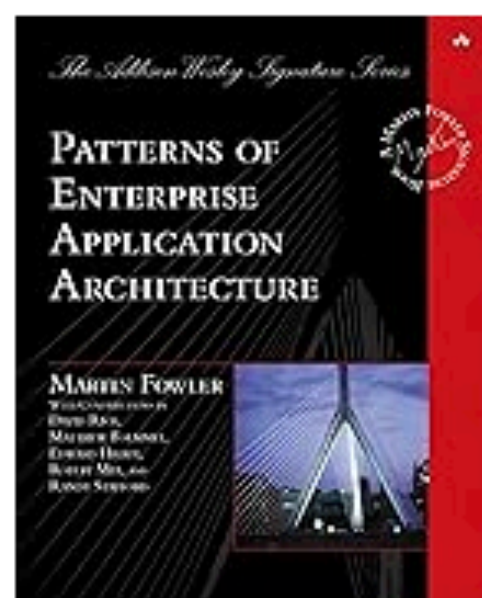
Initial state: { "quantity": 1 }

1. Renders "1"
2. User clicks "+" button
3. Change state to { "quantity": 2 }
4. Renders "2"

Shopping Cart

[Deselect all items](#)

Price



Patterns of Enterprise Application Architecture

by Martin Fowler

Hardcover

In Stock

[FREE International Returns](#)

☐ This is a gift [Learn more](#)

Format: Hardcover

☒ 1 + [Delete](#) [Save for later](#) [Compare with similar items](#) [Share](#)

\$51⁸⁰



The Missing README: A Guide for the New Software Engineer

by Chris Riccomini

Paperback

In Stock

[FREE International Returns](#)

☐ This is a gift [Learn more](#)

Format: Paperback

☒ 1 + [Delete](#) [Save for later](#) [Compare with similar items](#) [Share](#)

\$29²⁹

Subtotal (2 items): **\$81.09**

☐ This order contains a gift

[Proceed to checkout](#)



Enjoy fast, **FREE** delivery on eligible items with a 30-day trial for **\$0**

[Try Prime FREE](#)

Books you may like



[Project Hail Mary: A...](#)

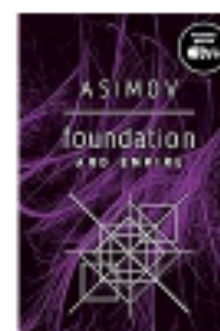
> Andy Weir

★★★★☆ 231,784

Kindle Edition

1 offer from **\$13⁹⁹**

[See all buying options](#)



[Foundation and Empire](#)

> Isaac Asimov

★★★★☆ 14,373

Kindle Edition

-5% \$8⁰⁵

Digital List Price: ~~\$8.48~~

[See all buying options](#)

Subtotal (2 items): **\$81.09**

Exercise

Identify stateful components

- CardItem
- ...

Identify stateful components

- CardItem
- Subtotal (2x)
- CartIndicator

Component decomposition

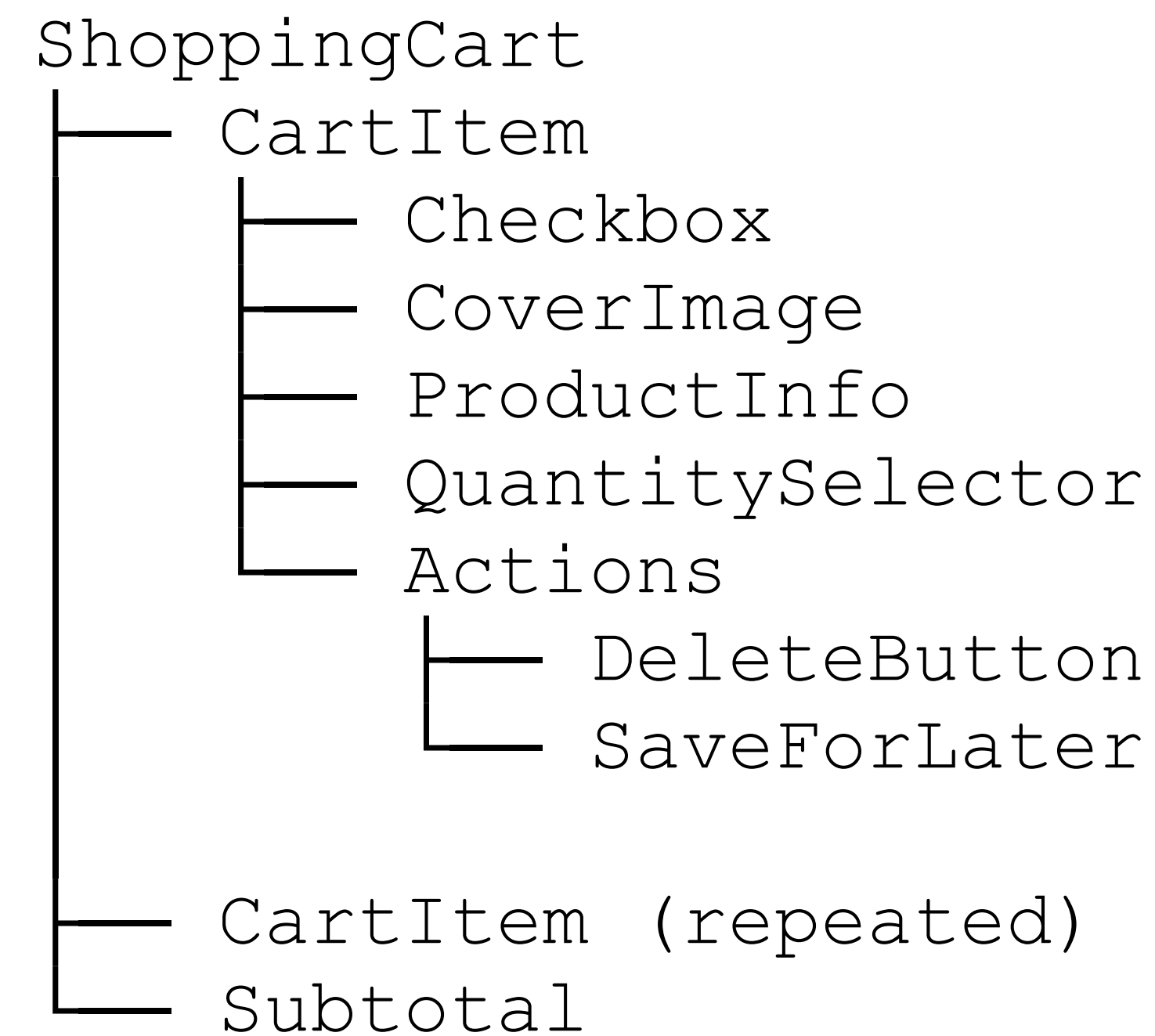
Composability

Shopping Cart

[Deselect all items](#)

		Price
☒	 Patterns of Enterprise Application Architecture by Martin Fowler Hardcover In Stock FREE International Returns ☐ This is a gift Learn more Format: Hardcover 🗑 1 + Delete Save for later Compare with similar items Share	\$51⁸⁰
☒	 The Missing README: A Guide for the New Software Engineer by Chris Riccomini Paperback In Stock FREE International Returns ☐ This is a gift Learn more Format: Paperback 🗑 1 + Delete Save for later Compare with similar items Share	\$29⁹⁹
Subtotal (2 items):		\$81.79

A component can be built from smaller (child) components

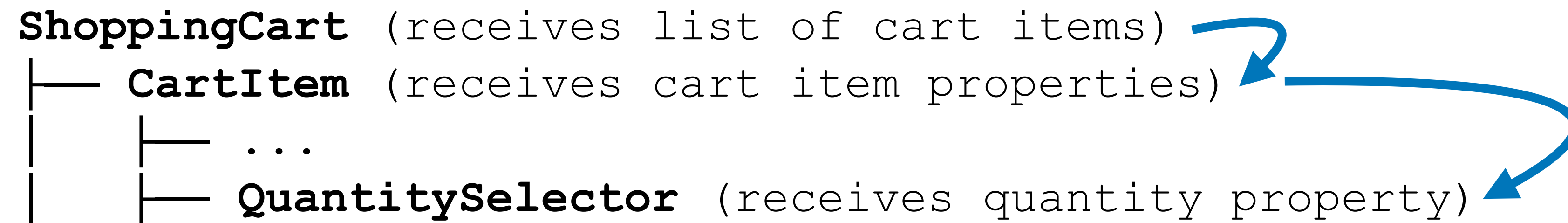


Composability lets us build complex UIs from simple, reusable building blocks

Composability

Props drilling problem

Composability makes it harder to pass properties/state to components



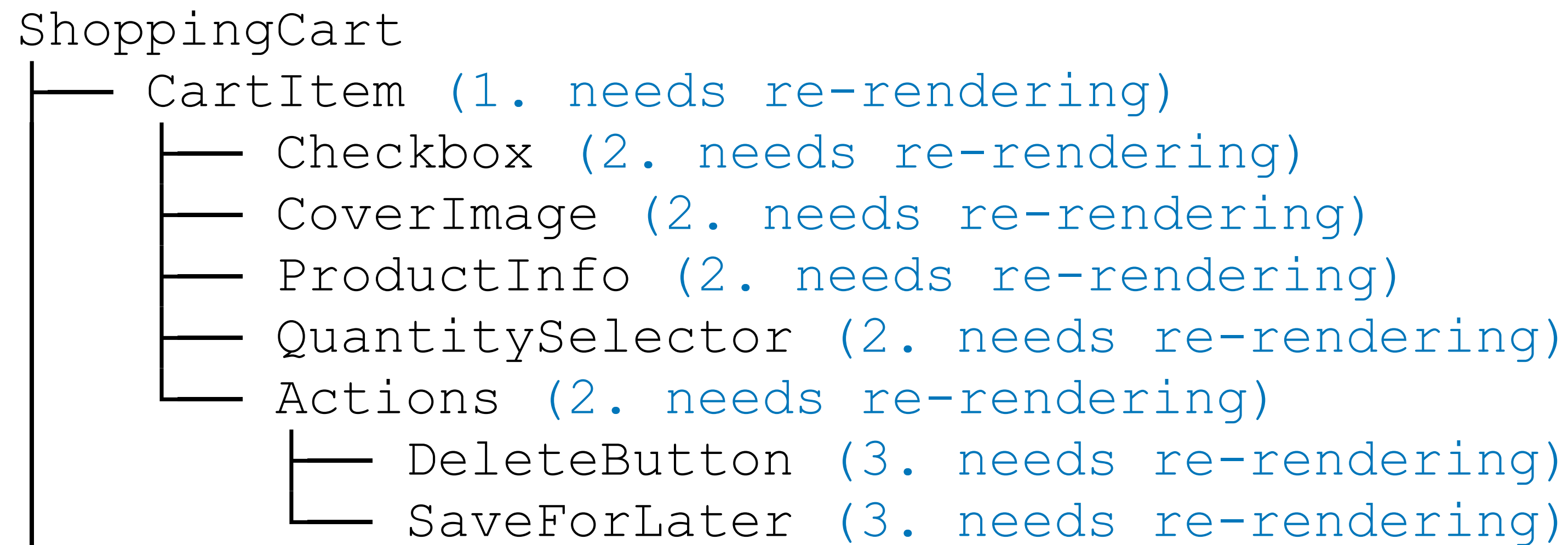
Problems:

- Intermediate components may not need the data
- Code becomes harder to maintain

Composability

Tree rendering

Rendering a component implies rendering all its children

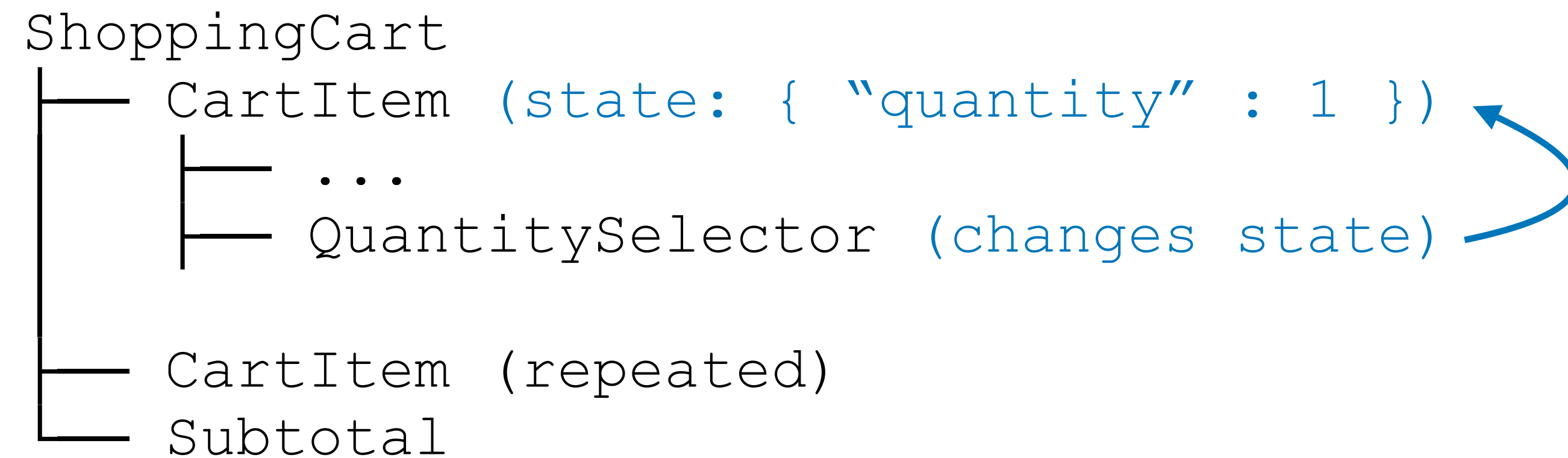


Problems:

- May lead to unnecessary renderings (components didn't really changed)
- Performance issues

State propagation

The component that **owns the state** is not always the one that **changes the state**

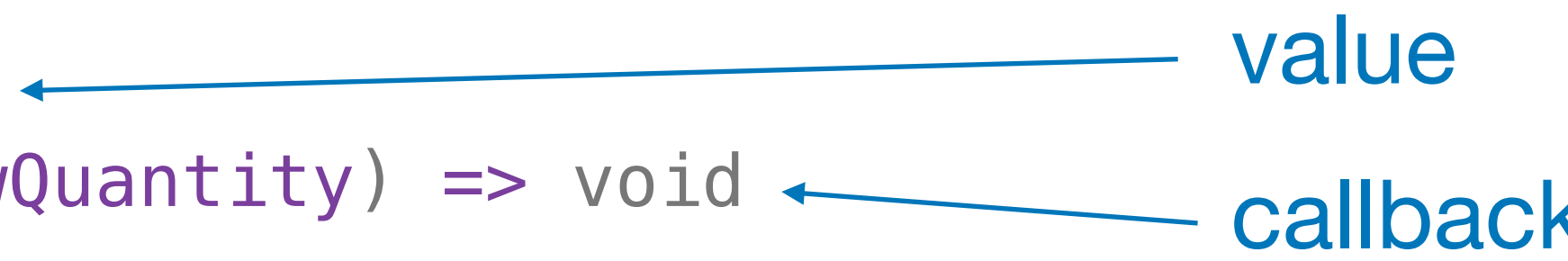


Composability

State propagation

1. CartItem **passes props** to QuantitySelector:

```
{  
  quantity : 1,  
  onChange: (newQuantity) => void  
}
```



value

callback

```
ShoppingCart  
├── CartItem  
│   ├── ...  
│   └── QuantitySelector  
├── CartItem (repeated)  
└── Subtotal
```

2. QuantitySelector **passes new value “calling back”** onChange in CartItem:

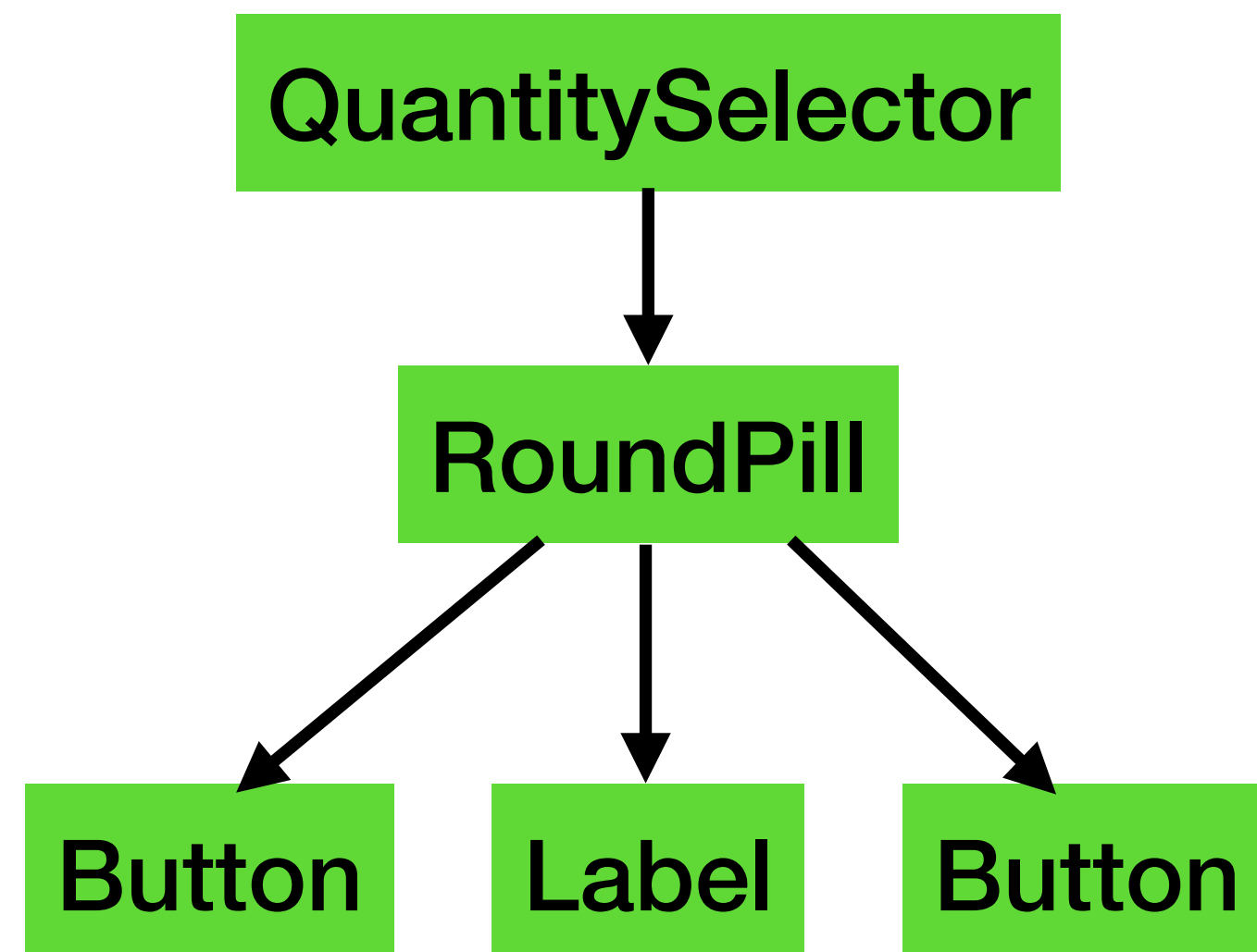
```
onChange: (+1) -> CartItem
```

Props flow down, events flow up

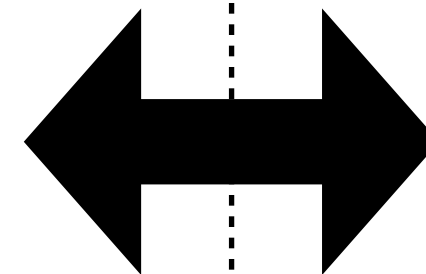
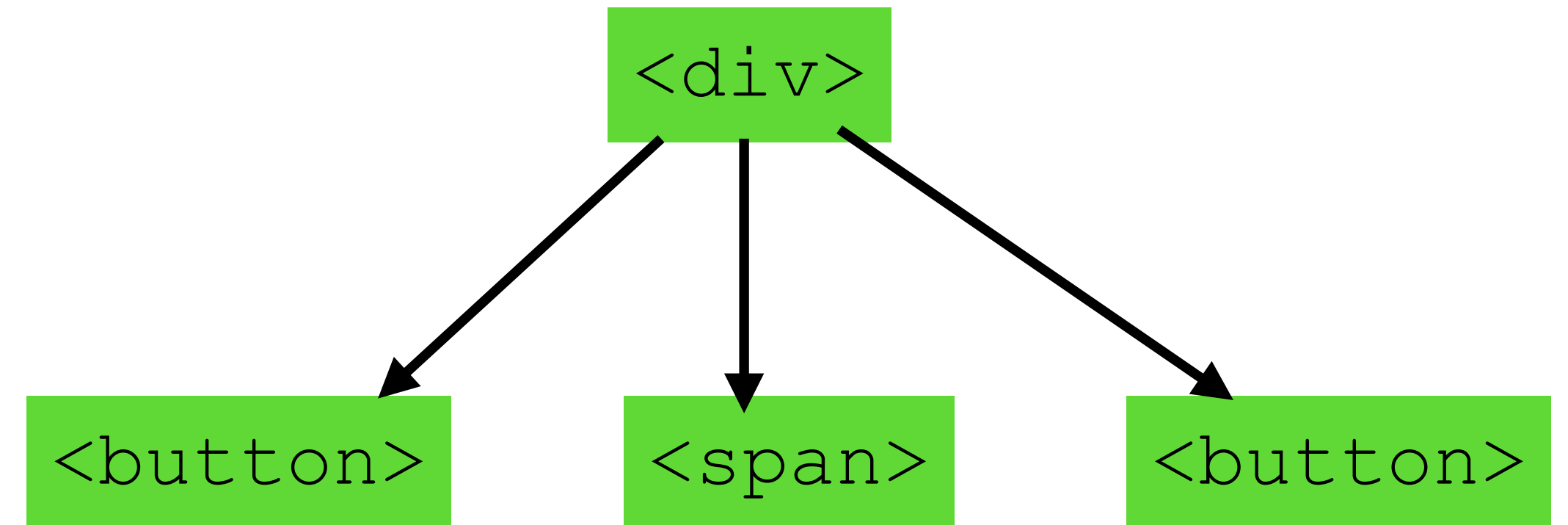
Component rendering



Component Tree (*)



DOM Tree



(*) Also called Virtual DOM

Javascript Dependency Management

npm (node package manager)

Similar to maven (instead of pom.xml, uses package.json)

```
{
  "name": "react-client",
  "version": "0.0.1",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },

  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.22.3"
  },

  "devDependencies": {
    "@types/react": "^18.3.1",
    "@types/react-dom": "^18.3.1",
    "@vitejs/plugin-react": "^4.3.1",
    "typescript": "^5.4.5",
    "vite": "^5.2.0"
  }
}
```

commands you can run with `npm run`

“production” dependencies

install dependencies
using `npm install`

“development” dependencies, not included in production

Javascript Dependency Management

In **maven**, by default all dependencies are stored in a single folder that is used by all projects (dependencies are shared)

windows: `C:\Users\YourUser\.m2\repository`

mac/linux: `/home/your-user/.m2/repository`

In **npm**, dependencies are stored per project into the `node_modules` folder. Dependencies are duplicated across projects

Alternative: **pnpm** - installs dependencies in a single folder; `node_modules` of each project have symlinks to that folder

```
npm install -g pnpm
```


Javascript Build Tools

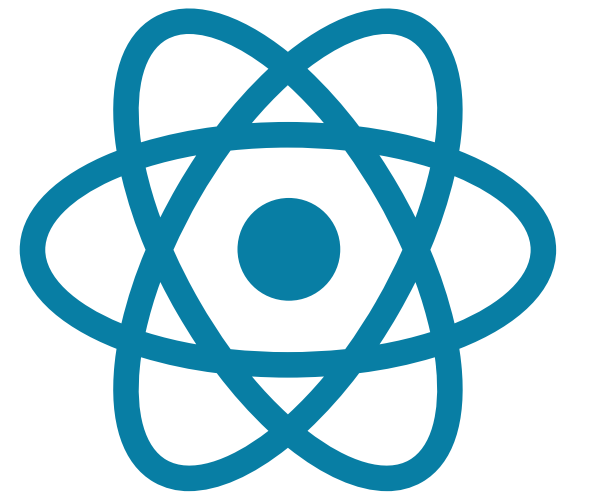
Browsers lack support for complex Javascript projects

- **Module bundling** - Combine many files into optimized bundles
- **Transpilation** - Convert alternative syntaxes (Typescript, JSX, ...) into compatible Javascript
- **Dependency resolution** - Browser alone cannot locate npm packages
- **Hot Reload** - Instant updates during development
- **Optimization for production** - minification, code splitting, compression, ...

Vite is a popular build tool that implements these features - <https://vite.dev/>

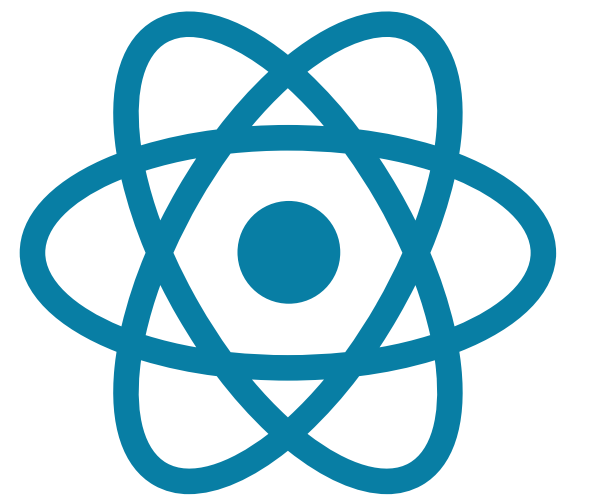
React

React supports the development of component-based web pages



- The user interface is a composition of predefined building blocks
- It allows to build encapsulated components that manage their own state
- Components can be assembled/composed to make more elaborate UIs
- Hierarchical structures are easily instantiated in many different contexts

React



Key features

- The view is built using JSX (HTML inside Javascript)
- It creates a virtual DOM that synchronizes with the real DOM very efficiently
- Data can be passed between components using props
- It allows state management through React Hooks
- Often used to build SPAs (commonly with React Router)

React bootstrap

1. Create the skeleton of a react app, using vite and typescript

```
pnpm create vite my-app --template react-ts
```

2. Navigate to the folder

```
cd my-app
```

3. Install dependencies

```
pnpm install
```

4. Run the dev environment - vite starts an HTTP dev server and enables Hot Reload

```
pnpm dev
```

React bootstrap

React will create a “standard” CSR page

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>book-shopping-page</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```


React stateless components

React stateless components are plain typescript functions (*) that take input data (props) and return what to display



`React.ReactNode` is a node of the Virtual DOM. It can be a string, a number, etc... but it usually is a `JSX.Element`

(*) The function name must start with uppercase

React stateless components

Props should be immutable

Greeting.tsx

```
interface GreetingProps {  
  readonly name: string  
}  
  
export function Greeting(props: GreetingProps) {  
  return <h1>Hello, {props.name}!</h1>  
}
```

Receives the name through
a “props” object

Export the function

Returns JSX

Imports/Exports

Functions are private to the file they belong unless you export them

```
function Hello() {  
  return <h1>Hello</h1>;  
}
```

Can't be used in other files

```
export function Hello() {  
  return <h1>Hello</h1>;  
}
```

Named exports, can be imported like this:

```
import { Hello } from "./Hello";
```

```
export default function Hello() {  
  return <h1>Hello</h1>;  
}
```

Default exports, can be imported like this:

```
import Hello from "./Hello";
```

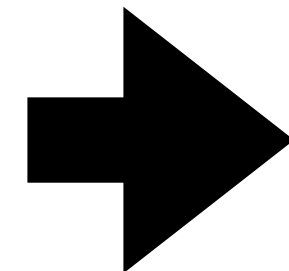
but also like this:

```
import HelloComponent from "./Hello";
```

JSX

JSX - JavaScript Syntax eXtension

```
<div>  
  <h1>Hello World!</h1>  
  <p>JSX code is awesome!</p>  
</div>
```



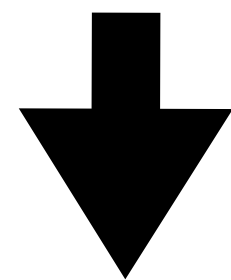
```
import { jsx } from "react/jsx-runtime";  
  
jsx("div", {  
  children: [  
    jsx("h1", { children: "Hello World!" }),  
    jsx("p", { children: "JSX code is awesome!" })  
  ]  
})
```

Used to describe and create HTML elements in Javascript in a declarative way

Browsers do not support JSX - Vite has a tool that transforms JSX code to plain Javascript

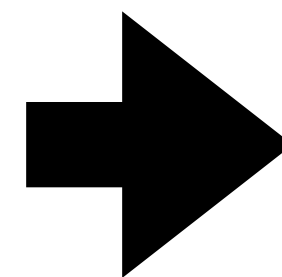
JSX

```
function Hello(): React.ReactElement {  
  return jsx("div", {  
    children: jsx("h1", { children: "Hello World!" })  
  })  
}
```



```
{  
  type: "div",  
  props: {  
    children: [  
      {  
        type: "h1",  
        props: {  
          children: "Hello World!"  
        },  
      },  
      {  
        type: "p",  
        props: {  
          children: "JSX code is awesome!"  
        },  
      },  
    ],  
  },  
}
```

React internally



```
const div = document.createElement("div");  
const h1 = document.createElement("h1");  
h1.textContent = "Hello World!";  
const p = document.createElement("p");  
p.textContent = "JSX code is awesome!";  
div.appendChild(h1);  
div.appendChild(p);
```


No JSX

You don't need to write JSX

```
function Hello() {  
  return React.createElement(  
    "div",  
    null,  
    React.createElement("h1", null, "Hello World!"),  
    React.createElement("p", null, "JSX code is awesome!")  
  );  
}
```

(Re)using Components

```
interface GreetingProps {  
  readonly name: string  
}
```

```
export function Greeting(props: GreetingProps) {  
  return <h1>Hello, {props.name}!</h1>  
}
```

Dynamic content should
be put inside { ... }

```
export default function App() {  
  return (  
    <>  
    <Greeting name="Pedro" />  
    <Greeting name="Sara" />  
    </>  
  )  
}
```

React will transform all
attributes into a Props object

JSX Elements must have a single root -
use fragments (<>)

Configuring Component Composition

```
import type {ReactNode} from "react";

interface BodyProps {
  readonly children: ReactNode
}

export default function Body(props: BodyProps): ReactNode {
  return (
    <div>
      {props.children}
    </div>
  );
}
```

```
export default function App() {
  return (
    <>
      <Header title="My App" />
      <Body>
        <Panel text="This is panel 1" />
        <Panel text="This is panel 2" />
      </Body>
    </>
  )
}
```

React will transform the
element content into
props.children object

Event Handling in React

Vanilla Javascript event handling

```
document.querySelector("button").addEventListener("click", () => {  
    alert("Button clicked!");  
});
```

React event handling

```
export default function Panel(props: Props) {  
  
    function handleClick() {  
        alert("Button clicked!");  
    }  
  
    return (  
        <div>  
            <p>{props.text}</p>  
            <button onClick={handleClick}>Click me</button>  
        </div>  
    )  
}
```

- declarative event binding
- component-scoped behaviour

Organizing Components

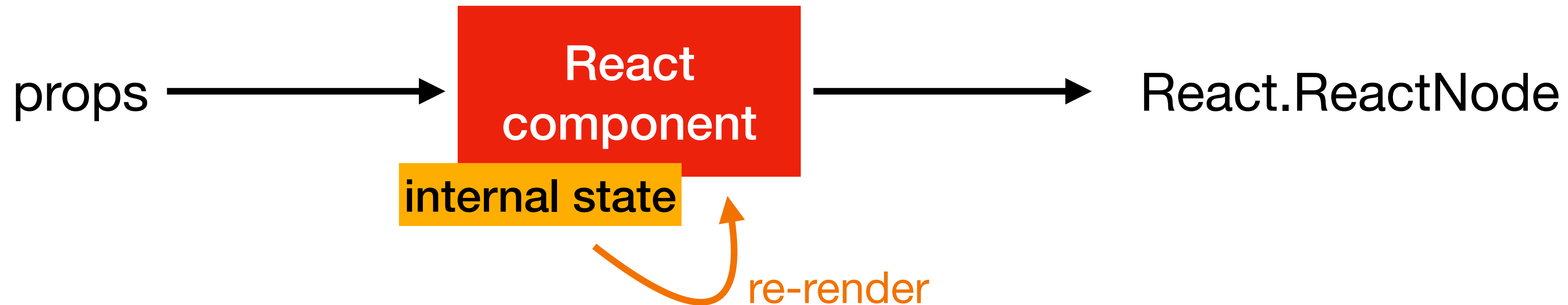
```
✓ src
  > assets
  ✓ components
    🌸 Body.tsx
    🌸 Header.tsx
    🌸 App.tsx
```

```
import Header from "../components/Header";
import Body from "../components/Body";

export default function App() {
  return (
    <>
      <Header title="My App" />
      <Body />
    </>
  )
}
```

React Stateful Components

React stateful components are plain typescript functions (*) that take input data (props), **manage internal state**, and return what to display



If the internal state changes, the component is re-rendered

React Stateful Components

This is a stateless component, it is only rendered once

```
import styles from "../QuantitySelector.module.css";
import {useState} from "react";

export default function QuantitySelector(): React.ReactNode {

  let quantity = 1;
  console.log(`Rendering QuantitySelector: ${quantity}`);

  return (
    <div className={styles.container}>
      <button className={styles.btn} onClick={() => {
        const newQuantity = quantity - 1;
        console.log("Clicked -. Quantity is now " + quantity + ".")
        quantity = newQuantity
      }}>-</button>
      <span className={styles.quantity}>{quantity}</span>
      <button className={styles.btn} onClick={() => {
        const newQuantity = quantity + 1;
        console.log("Clicked +. Quantity is now " + quantity + ".")
        quantity = newQuantity
      }}>+</button>
    </div>
  );
}
```

- 1 +

Elements Console Recorder Sources >>	
top Filter Default levels No Issues	
2 hidden	
Rendering QuantitySelector: 1	QuantitySelector.tsx:8
Rendering QuantitySelector: 1	installHook.js:1
Clicked +. Quantity is now 1.	QuantitySelector.tsx:20
Clicked +. Quantity is now 2.	QuantitySelector.tsx:20
Clicked +. Quantity is now 3.	QuantitySelector.tsx:20
Clicked +. Quantity is now 4.	QuantitySelector.tsx:20

It doesn't work as expected

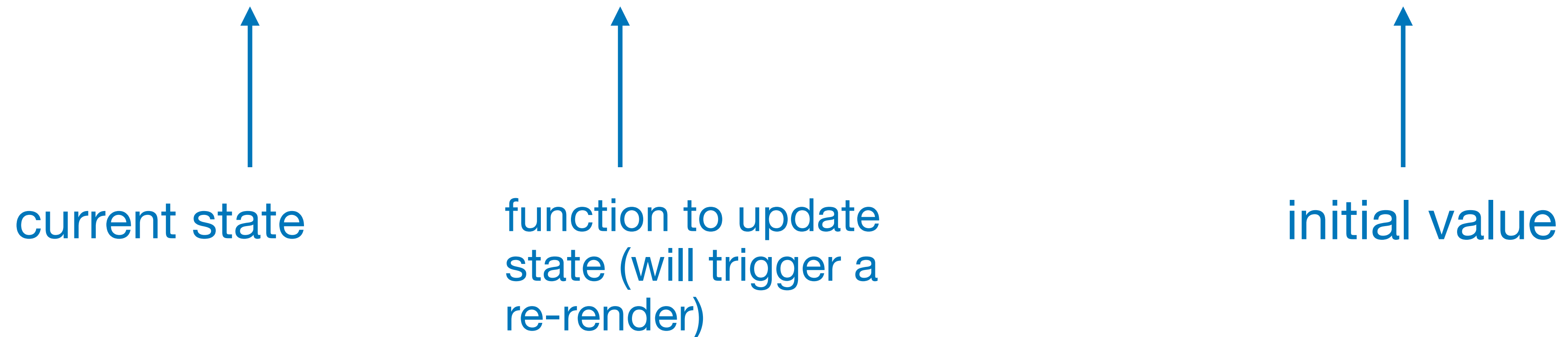
React Stateful Components

State must be stored outside the component

React provides a hook to access and set state - `useState`

```
const [counter, setCounter] = useState<number>(0);
```

current state



function to update
state (will trigger a
re-render)

initial value

React Stateful Components

This is a stateful component, re-rendered on every `setQuantity`

```
import styles from './QuantitySelector.module.css';
import {useState} from 'react';

export default function QuantitySelector(): React.ReactNode {

  const [quantity, setQuantity] = useState<number>(1); ←
  console.log(`Rendering QuantitySelector: ${quantity}`);

  return (
    <div className={styles.container}>
      <button
        className={styles.btn}
        onClick={() => {
          const newQuantity = quantity - 1;
          console.log("Clicked -. Quantity is now " + newQuantity + ".")
          setQuantity(newQuantity) ←
        }}
      >-</button>
      <span className={styles.quantity}>{quantity}</span>
      <button
        className={styles.btn}
        onClick={() => {
          const newQuantity = quantity + 1;
          console.log("Clicked +. Quantity is now " + newQuantity + ".")
          setQuantity(newQuantity) ←
        }}
      >+</button>
    </div>
  );
}
```

- 3 +

Elements

Console

Recorder

Sources

>>

top ▾

Filter

Default levels ▾

No Issues

2 hidden

Rendering QuantitySelector: 1

[QuantitySelector.tsx:8](#)

Rendering QuantitySelector: 1

[installHook.js:1](#)

Clicked +. Quantity is now 2.

[QuantitySelector.tsx:25](#)

Rendering QuantitySelector: 2

[QuantitySelector.tsx:8](#)

Rendering QuantitySelector: 2

[installHook.js:1](#)

Clicked +. Quantity is now 3.

[QuantitySelector.tsx:25](#)

Rendering QuantitySelector: 3

[QuantitySelector.tsx:8](#)

Rendering QuantitySelector: 3

[installHook.js:1](#)

Exercise

```
import {useState} from "react";

export function CounterExercise() {

  const [count, setCount] = useState(0);

  function handleClick() {
    console.log("A:", count);
    setCount(count + 1);
    console.log("B:", count);
  }

  console.log("RENDER:", count);

  return (
    <button onClick={handleClick}>
      Count is {count}
    </button>
  );
}
```

Console output after clicking
the button twice?

Resolution

```
import {useState} from "react";

export function CounterExercise() {

  const [count, setCount] = useState(0);

  function handleClick() {
    console.log("A:", count);
    setCount(count + 1);
    console.log("B:", count);
  }

  console.log("RENDER:", count);

  return (
    <button onClick={handleClick}>
      Count is {count}
    </button>
  );
}
```

Console output after clicking
the button twice?

RENDER: 0
A: 0
B: 0
← first button click

RENDER: 1
A: 1
B: 1
← second button click

RENDER: 2

Objects as State

Example: setting the name of a user

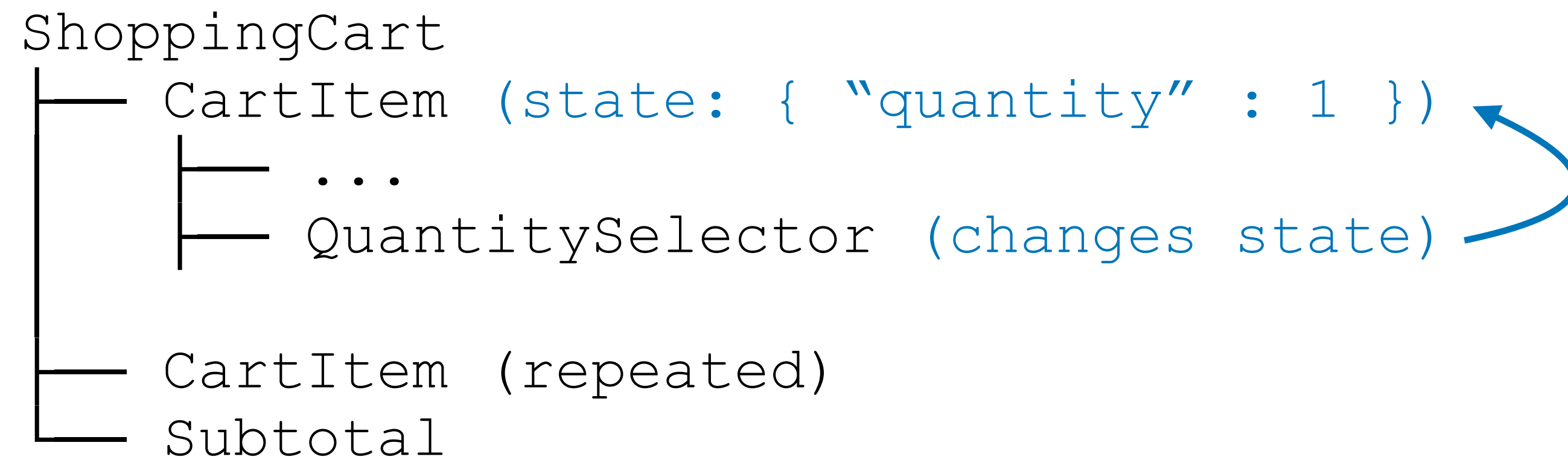
```
interface User {  
  readonly name: string;  
  readonly age: number;  
}  
  
export default function Profile() {  
  const [user, setUser] = useState<User>({  
    name: "Alice",  
    age: 25,  
  });  
  
  function handleUpdateName() {  
    setUser({ ...user, name: "Bob" })  
  }  
  
  return (  
    <div>  
      <p>{user.name}</p>  
      <p>{user.age}</p>  
      <button onClick={handleUpdateName}>Change name to Bob</button>  
    </div>  
  );  
}
```

React favours immutability: you have to replace the whole object

... is the spread operator - copy everything from user, override name field

State propagation

The component that **owns the state** is not always the one that **changes the state**



Props flow down, events flow up

React

State propagation

```
interface QuantitySelectorProps {  
  readonly quantity: number;  
  readonly onChange: (newQuantity: number) => void;  
}  
  
export default function QuantitySelector({ quantity, onChange }: QuantitySelectorProps) {  
  
  return (  
    <div className={styles.container}>  
      <button className={styles.btn}  
        onClick={() => onChange(quantity-1)}  
      >-</button>  
      <span className={styles.quantity}>{quantity}</span>  
      <button className={styles.btn}  
        onClick={() => onChange(quantity+1)}  
      >+</button>  
    </div>  
  );  
}
```

receives the current state

receives a callback to “flow up” the event

React

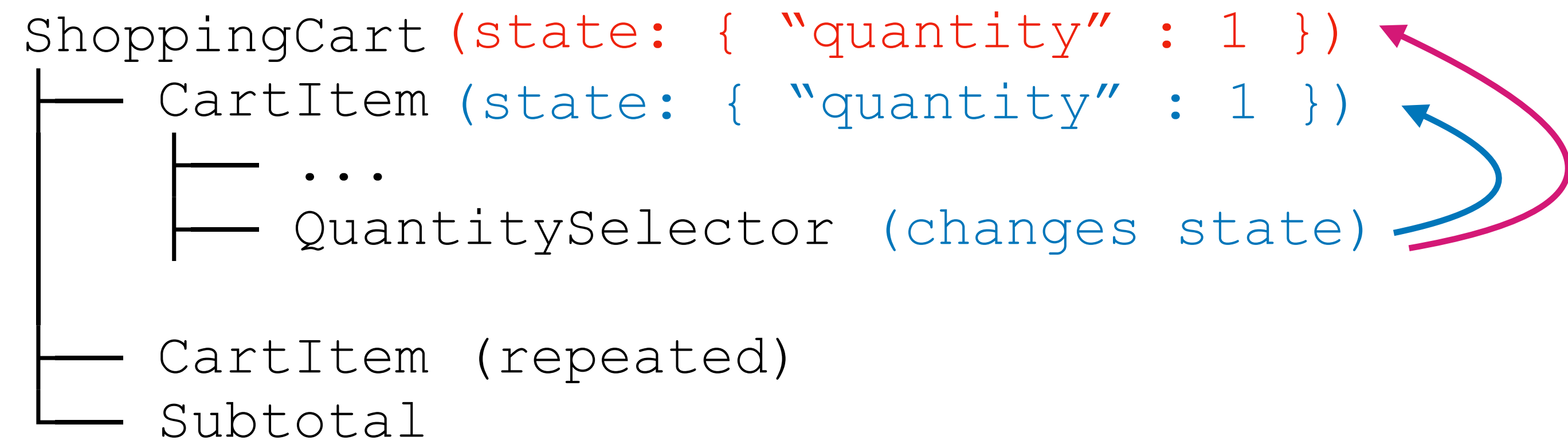
State propagation

```
export default function CartItem(props: CartItemModel) {  
  
  const [quantity, setQuantity] = useState<number>(props.quantity);  
  
  return (  
    <div>  
      ...  
      <QuantitySelector  
        quantity={quantity}  
        onChange={(newQuantity) => setQuantity(newQuantity)} />  
    </div>  
  );  
}
```

Re-render of the parent
-> re-render its children

```
export default function QuantitySelector({ quantity, onChange }: QuantitySelectorProps) {  
  
  ...  
  <button onClick={() => onChange(quantity+1)}>+</button>  
  
}
```


State propagation



This component also needs to know/react to the quantity

Rule of thumb: State should live in the closest common parent component

State propagation

```
export default function ShoppingCart() {  
  const [items, setItems] = useState<CartItemModel[]>(initialItems);  
  
  function handleQuantityChange(id: string, newQuantity: number) {  
    setItems(...);  
  }  
  
  <CartItem {...item} onQuantityChange={(qty) => handleQuantityChange(item.id, qty)} />  
  ...  
  <Subtotal items={items} />  
}  
  
export default function CartItem(props: CartItemProps) {  
  return (  
    ...  
    <QuantitySelector quantity={props.quantity}  
      onChange={props.onQuantityChange} />  
    ...  
  );  
}  
  
export default function QuantitySelector(  
  { quantity, onChange }: QuantitySelectorProps): ReactNode {  
  
  <button onClick={() => onChange(quantity-1)}>-</button>  
  
}
```

The diagram illustrates the flow of state updates through three components:

- ShoppingCart**: Contains the `useState` hook for `items` and a `handleQuantityChange` function that calls `setItems(...)`.
- CartItem**: Receives `onQuantityChange` as a prop and passes it to `QuantitySelector`.
- QuantitySelector**: Contains a `button` that calls `onChange(quantity-1)` when clicked.

Blue arrows indicate the propagation path: from `handleQuantityChange` in `ShoppingCart` to `setItems(...)`, from `onQuantityChange` in `CartItem` to `handleQuantityChange` in `ShoppingCart`, and from `onChange` in `QuantitySelector` to `onQuantityChange` in `CartItem`.

React Exercise

Spot the errors

```
1. interface CardProps {
2.     title: string;
3.     children: React.ReactNode;
4. }
5.
6. function card(props: CardProps) {
7.     return (
8.         <h2>{props.title}</h2>
9.         {props.children}
10.    )
11. }
12.
13. function Button(props: { content: string }) {
14.     return <button>{content}</button>;
15. }
16.
17. export default function App() {
18.     return (
19.         <card title={123}>
20.             <Button>
21.                 Click me
22.             </Button>
23.         </card>
24.     );
25. }
```


React Resolution

Components must start with uppercase

Components must return a single element (use fragments)

Should be props.content

Missing content attribute

Ignored since button props don't include children

```
1. interface CardProps {
2.     title: string;
3.     children: React.ReactNode;
4. }
5.
6. function card(props: CardProps) {
7.     return (
8.         <h2>{props.title}</h2>
9.         {props.children}
10.     )
11. }
12.
13. function Button(props: { content: string }) {
14.     return <button>{content}</button>;
15. }
16.
17. export default function App() {
18.     return (
19.         <card title={123}>
20.             <Button>
21.                 Click me
22.             </Button>
23.         </card>
24.     );
25. }
```